

FitApp

Documentazione Tecnica

Giulio Dini

1 Sommario

2	Raccolta dei Requisiti.....	3
2.1	Requisiti Funzionali.....	3
2.2	Requisiti non Funzionali.....	3
2.3	Requisiti Tecnici	3
3	Modellazione Funzionale.....	5
4	Progettazione Statica.....	6
4.1	Diagramma delle Classi.....	6
5	Progettazione Dinamica	7
5.1	Diagramma di Sequenza	7

2 Raccolta dei Requisiti

La specifica dei requisiti adotta l'approccio standard dell'ingegneria del software, separando le funzionalità esplicite del sistema (Requisiti Funzionali) dai vincoli qualitativi e tecnologici (Requisiti Non Funzionali). Questa categorizzazione formale stabilisce un perimetro chiaro per lo sviluppo di FitApp, garantendo la tracciabilità tra le richieste dell'utente e l'implementazione del codice.

2.1 Requisiti Funzionali

Codice	Titolo	Descrizione	Priorità
RF-01	Gestione Peso	L'utente deve poter inserire una nuova misurazione del peso specificando valore (kg) e data.	Alta
RF-02	Storico Pesate	Il sistema deve mostrare lo storico delle pesate dell'utente sotto forma di grafico temporale.	Alta
RF-03	Diario Alimentare	L'utente deve poter inserire gli alimenti consumati nella giornata, catalogandoli per pasto (Colazione, Pranzo, Cena, Merende).	Alta
RF-04	Calcolo Macro	Il sistema deve calcolare automaticamente in tempo reale il totale di Calorie, Proteine, Carboidrati e Grassi assunti nella giornata.	Alta
RF-05	Gestione Schede	L'utente deve poter creare una nuova scheda di allenamento assegnandole un nome e una descrizione.	Alta
RF-06	Gestione Esercizi	L'utente deve poter aggiungere, eliminare o riordinare gli esercizi all'interno di una specifica scheda.	Alta
RF-07	Avanzamento Allenamento	L'utente deve poter smarcare un esercizio come "Completato" e visualizzare la barra di avanzamento della scheda in tempo reale.	Media
RF-08	Calcolo BMI	Il sistema deve calcolare il BMI (Indice di Massa Corporea) e lo stato ponderale partendo dall'altezza e dall'ultimo peso inserito.	Media

2.2 Requisiti non Funzionali

- **RNF-01 (Usabilità):** L'interfaccia utente deve essere reattiva (Responsive), adattandosi sia a schermi Desktop che a dispositivi Mobile.
- **RNF-02 (Prestazioni):** Il passaggio di stato o il calcolo dei macro giornalieri sul frontend deve avvenire in modo immediato tramite gestione asincrona dei dati, senza richiedere il rinfresco (refresh) della pagina del browser.
- **RNF-03 (Manutenibilità):** Il codice deve essere strutturato seguendo l'architettura a strati disaccoppiati (Separation of Concerns) per consentire modifiche future a un singolo livello senza impattare sugli altri.
- **RNF-04 (Consistenza):** Ogni operazione di modifica (digitazione note, cambio carico, spunta completato) deve essere sincronizzata immediatamente con il database per evitare perdite di dati in caso di chiusura improvvisa dell'applicazione.

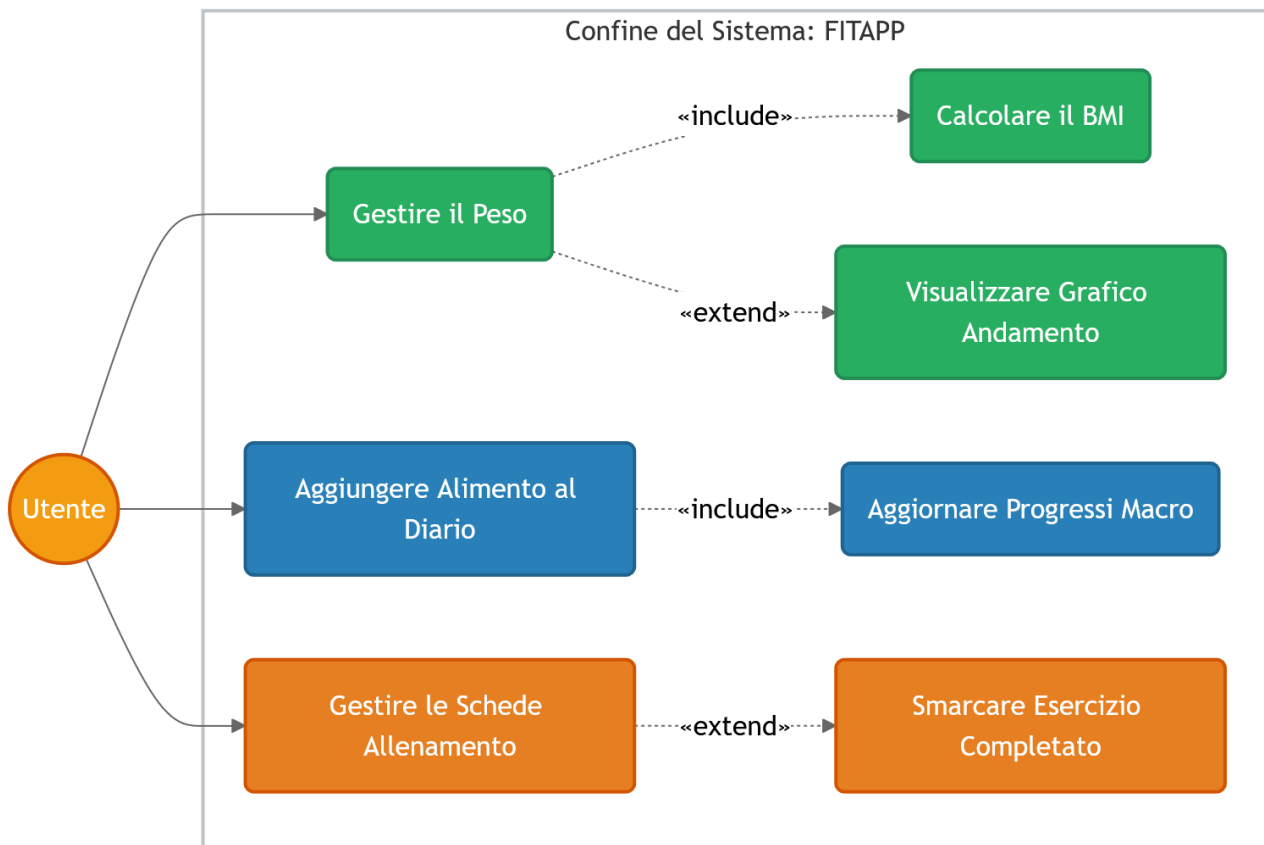
2.3 Requisiti Tecnici

- **RT-01 (Frontend):** Sviluppo dell'interfaccia client basato sul framework Angular 17+, sfruttando l'architettura a Componenti e la gestione dello stato tramite Signals.
- **RT-02 (Backend):** Sviluppo delle logiche di business e delle API basato su Java e il framework Spring Boot 3+.

- **RT-03 (Data Access):** Utilizzo dello standard JPA con Hibernate per l'Object-Relational Mapping (ORM).
- **RT-04 (Database):** Utilizzo di un database relazionale H2 integrato (configurato inizialmente in memoria o file locale) per la persistenza dei dati.
- **RT-05 (Comunicazione):** Lo scambio dati tra client e server deve avvenire esclusivamente tramite protocollo HTTP/REST con payload in formato JSON.

3 Modellazione Funzionale

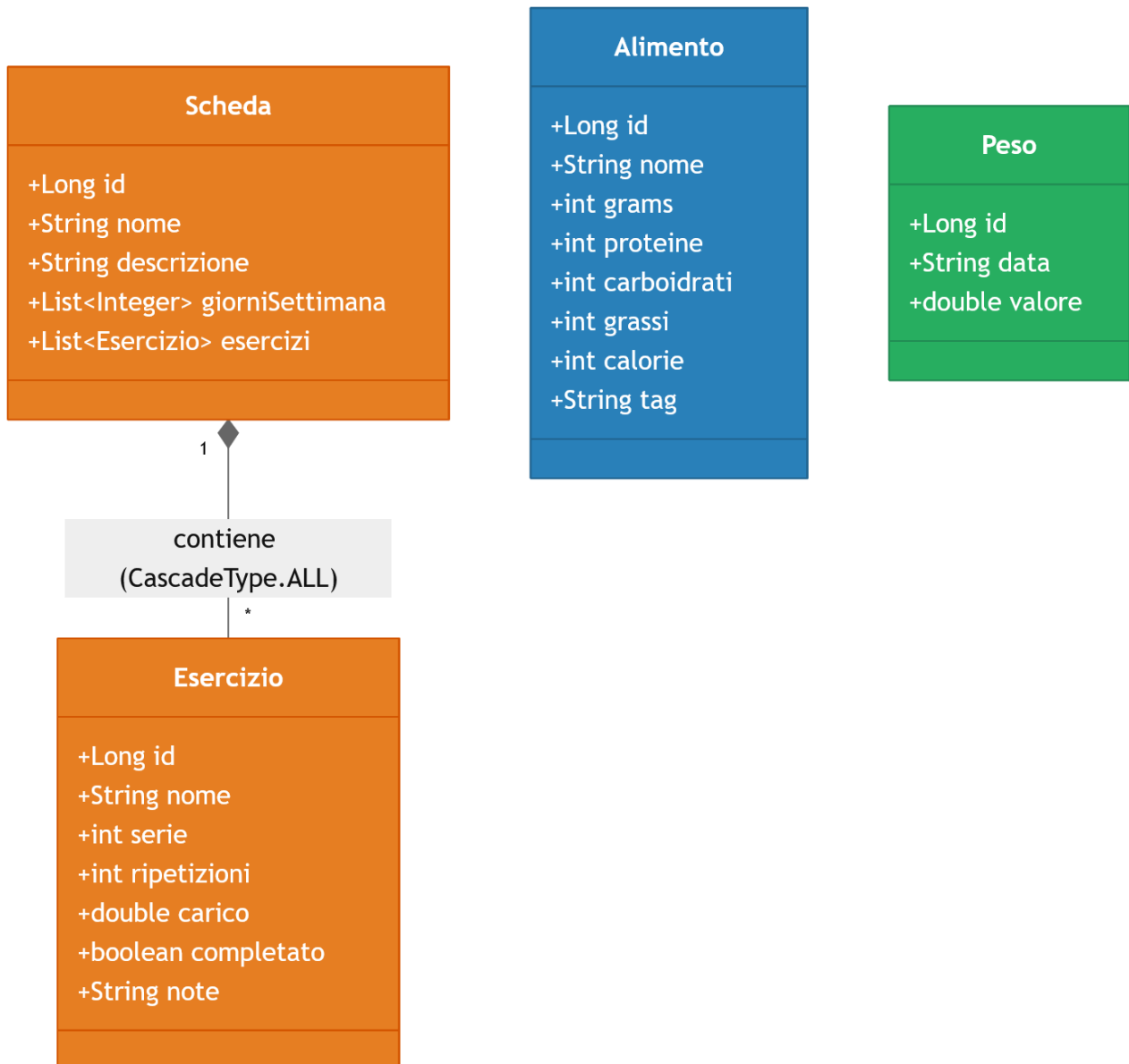
Il Diagramma dei Casi d'Uso descrive le interazioni macroscopiche tra l'Utente e i confini del sistema FitApp, isolando le funzioni in livelli logici di profondità attraverso le relazioni di inclusione ed estensione. L'uso delle relazioni "include" evidenzia come operazioni elementari inneschino automatismi di calcolo immediati (come l'aggiornamento dei macro o del BMI), mentre la relazione "extend" mappa le azioni opzionali, riflettendo accuratamente il comportamento dell'interfaccia utente senza esporre i dettagli del codice.



4 Progettazione Statica

4.1 Diagramma delle Classi

Il Diagramma delle Classi definisce la struttura logica permanente dei dati e mappa specularmente il modello a oggetti condiviso tra i tipi TypeScript del Frontend e le entità JPA del database H2. La relazione di Composizione (identificata dal rombo pieno) tra la classe Scheda e la classe Esercizio è il perno dell'area allenamento: essa esprime un vincolo di dipendenza forte in cui gli esercizi non possono esistere in modo autonomo, legando il loro ciclo di vita a quello della scheda tramite meccanismi di persistenza a cascata.



5 Progettazione Dinamica

5.1 Diagramma di Sequenza

Il Diagramma di Sequenza illustra la natura dinamica e asincrona dell'applicazione, tracciando il ciclo di vita di una richiesta dall'evento di focus-out (blur) sul browser fino alla persistenza su disco. Il flusso evidenzia l'efficacia del disaccoppiamento architetturale (Separation of Concerns): il client Angular comunica in modo non bloccante tramite l'**Observer Pattern**, mentre il server Spring Boot metabolizza il dato attraversando rigidamente lo strato Controller, il **Service Layer** per le validazioni di business, e lo strato Repository per la traduzione in query SQL native.

